

Absence Request Demo

Every block of code explained - three files, 336 lines

FastAPI + SMTP with OAuth 2.0 (Google Cloud) + Jinja2 HTML email

```
demo_smtp/
  app/
    store.py      86 lines  the data + the approved list
    mail.py       129 lines  SMTP with an OAuth 2.0 login + the two emails
    main.py       121 lines  the four HTTP routes
  approved.json   who may ask (employees) and approve (managers)
  templates/      the HTML emails + the manager's page
  check_gmail.py  one-time Google authorization
```

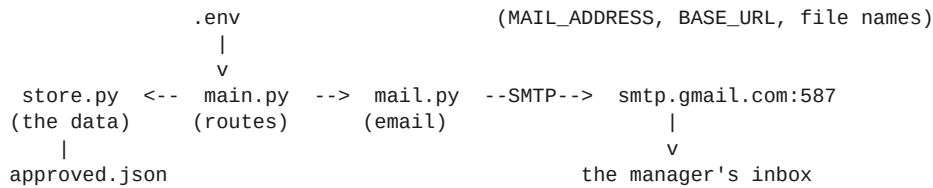
The flow:

1. employee POST /nghiphep -> checked, saved PENDING, manager emailed
2. manager clicks Yes or No -> GET /nghiphep/{id}/decision
3. system saves the answer -> APPROVED or REJECTED
4. system emails the employee -> and shows the manager a small page

Read the files in this order. `store.py` depends on nothing, `mail.py` uses only settings, and `main.py` uses both.

0. How the three files fit together

One page of orientation before any code.



Each file has exactly one job:

- `store.py` - remembers requests. Never sends anything, never touches HTTP.
- `mail.py` - builds the two emails and sends them over SMTP, logging in with a Google token.
- `main.py` - the web API. Decides what happens, then asks the other two to do it.

Nothing imports `main.py`, and `store.py` imports nothing from this project at all. That is what makes the code easy to follow: there are no cycles, so you can read it straight down.

The one idea behind the whole design

There is **no login**. The manager proves who they are by holding a secret that only the email contains:

<code>store.py</code>	creates a random token for each request
<code>mail.py</code>	puts that token inside the [Yes] and [No] links
<code>main.py</code>	checks the token when somebody clicks
<code>main.py</code>	never returns the token in any API response

If the API returned the token, anybody who could create a request could immediately approve it. The token exists only inside the manager's email. That single rule is worth stating clearly in the seminar.

1. store.py

The data. One dictionary in memory - no database.

Block 1 - the lock and the dictionary

```
# Uvicorn handles requests in several threads, so two clicks can arrive together.
_lock = threading.Lock()

# Who may ask and who may approve. No file = everyone is allowed.
APPROVED_FILE = Path(__file__).resolve().parent.parent / "approved.json"

_requests = {} # id -> AbsenceRequest
_next_id = 1
```

One dictionary standing in for a database table, and a counter playing the part of an auto-increment id. The leading underscore is the Python convention for *private* - other files go through the functions, never the dictionary.

The lock is not decoration. Uvicorn runs ordinary endpoint functions in a thread pool, so two requests really can run at the same instant. Everything that writes to the dictionary takes the lock first. Block 4 shows why that matters.

Block 2 - what a request looks like

```
@dataclass
class AbsenceRequest:
    id: int
    employee_email: str
    manager_email: str
    reason: str
    from_date: str
    to_date: str
    status: str = "PENDING" # PENDING -> APPROVED or REJECTED
    token: str = field(default_factory=_new_token) # secret inside the Yes/No links
    created_at: datetime = field(default_factory=_now)
```

@dataclass writes `__init__` and a readable `__repr__` for you. You describe the fields; Python generates the rest.

`field(default_factory=_new_token)` **instead of** `_new_token()` is the subtle line. A plain default is evaluated *once*, when the class is defined - so **every request would share a single token**, and anyone with one link could approve everybody's requests. `default_factory` stores the function and calls it again for each new object.

```
def _new_token():
    return secrets.token_urlsafe(32)
```

32 bytes = 256 bits of randomness, written as URL-safe text so it can go straight into a link. Use `secrets`, never `random`: the `random` module is built for simulations and is predictable if you see enough of its output.

Block 3 - the approved list

```
def is_approved(email, role):
    if not APPROVED_FILE.exists():
        return True
    lists = json.loads(APPROVED_FILE.read_text(encoding="utf-8"))
    allowed = [e.strip().lower() for e in lists.get(role, [])]
    return email.lower() in allowed
```

Answers one question: *is this address on the list for this role?* `role` is "employees" or "managers", the two keys in `approved.json`:

```
{
    "employees": ["worker@example.com"],
```

```

"managers": ["boss@example.com"]
}

```

- **Two separate lists** - roles cannot be swapped. An employee cannot approve, and a manager cannot file a request, so nobody can name themselves as their own manager.
- **Lower-cased on both sides** - `Worker@Example.com` matches `worker@example.com`, because email addresses are case-insensitive.
- **Read on every call** - you can edit the list while the server runs; the next request already uses it.
- **No file means everyone is allowed** - so a first run works without extra setup.

Block 4 - saving and reading

```

def add(employee_email, manager_email, reason, from_date, to_date):
    global _next_id
    with _lock:
        req = AbsenceRequest(_next_id, employee_email, manager_email,
                             reason, from_date, to_date)
        _requests[req.id] = req
        _next_id += 1
        return req

def get(request_id):
    return _requests.get(request_id)

```

`.get()` returns `None` for an unknown id instead of raising an error, so the caller can simply test for `None`. `global _next_id` is needed because the function assigns to it; reading alone would not require it.

Block 5 - decide: the heart of the file

```

def decide(request_id, approved):
    with _lock:
        req = _requests.get(request_id)
        if req is None or req.status != "PENDING":
            return None
        req.status = "APPROVED" if approved else "REJECTED"
        return req

```

This is the only function that changes an answer, and it is written so the change happens **at most once**.

Read it as **check, then act**: check the status is still `PENDING`, then write the new status. Without the lock, two clicks arriving together could both run the check before either ran the write - both would see `PENDING`, both would proceed, and the employee would get two result emails.

Returning `None` is the signal. `main.py` uses it to choose between "Request approved" and "Already answered". The second click changes nothing and sends no email.

2. mail.py

SMTP, an OAuth 2.0 login from Google Cloud, and the two emails.

This file sends email with **SMTP**, the standard email protocol. It connects to Gmail's SMTP server, encrypts the connection, and logs in with an **OAuth 2.0 token** instead of a password. The token comes from three artefacts:

credentials.json	who this app is	(downloaded from Google Cloud)
consent screen	the owner says "yes"	(once, in the browser)
token.json	proof of that "yes"	(created by check_gmail.py)

Google Cloud does not send the mail - Gmail's SMTP server does. Google Cloud is where the app was registered, so the mailbox owner could approve it. The resulting token replaces the password in the SMTP login. Hide token.json and the login fails: that is the demo worth doing live.

Block 1 - settings, server and scope

```
load_dotenv() # read the .env file, so no secret is written in the code

MAIL_ADDRESS = os.getenv("MAIL_ADDRESS", "").strip()
BASE_URL = os.getenv("BASE_URL", "http://localhost:8000").strip().rstrip("/")

# Gmail's SMTP server. Port 587 = start in plain text, then upgrade with STARTTLS.
SMTP_HOST = "smtp.gmail.com"
SMTP_PORT = 587

# Google only accepts OAuth logins over SMTP with this scope (full mail access).
SCOPES = ["https://mail.google.com/"]
```

Port 587 is the *submission* port: the connection starts in plain text and is upgraded to TLS with STARTTLS before anything secret is sent. (Port 465 would be encrypted from the first byte; port 25 is for server-to-server delivery.)

A **scope** is a named permission. For SMTP, Google only accepts OAuth logins with the full-mail scope `https://mail.google.com/`, which allows reading and deleting too. That is the price of using SMTP: the Gmail API version of this demo could ask for send-only (`gmail.send`). See the comparison page at the end.

Block 2 - the template environment

```
templates = Environment(
    loader=FileSystemLoader(Path(__file__).resolve().parent.parent / "templates"),
    autoescape=select_autoescape(["html"]),
)
```

autoescape is the security line. The employee's *reason* is written by a user and dropped into an HTML email. Without escaping, a reason of `<script>...` would be injected as live markup. Autoescaping turns `<` into `<`; so it is shown as text.

Block 3 - get_credentials: getting a usable token

```
def get_credentials(interactive=False):
    creds = None
    if Path(TOKEN_FILE).exists():
        saved = json.loads(Path(TOKEN_FILE).read_text())
        # A token made for another scope is useless here: ignore it and ask again.
        if set(SCOPES) <= set(saved.get("scopes", [])):
            creds = Credentials.from_authorized_user_info(saved, SCOPES)

    if creds and creds.valid:
        # 1. still good
        return creds

    if creds and creds.refresh_token:
        # 2. expired, but renewable
        creds.refresh(Request())
        Path(TOKEN_FILE).write_text(creds.to_json())
```

```

    return creds

if not interactive:
    # 3. we need a human
    raise RuntimeError("Gmail is not authorized. Run: python check_gmail.py")

flow = InstalledAppFlow.from_client_secrets_file(CREDENTIALS_FILE, SCOPES)
creds = flow.run_local_server(port=0, prompt="consent")

```

The scope check comes first. `set(SCOPES) <= set(...)` asks: does the saved token cover the scope we need? A token approved for something else (for example the send-only Gmail API scope) would be refused by the SMTP server, so it is ignored and the owner is asked again.

Then a ladder, cheapest rung first:

- **Case 1** - the normal path. An access token lasts about an hour.
- **Case 2** - the access token expired, but the long-lived **refresh token** buys a new one from Google with no human involved.
- **Case 3** - nothing usable. `check_gmail.py` passes `interactive=True` and may open a browser; the running server passes `False` and raises instead. **A web server must never block waiting for a browser window nobody is watching.**

Block 4 - send: the SMTP conversation

```

msg = EmailMessage()
msg["From"] = MAIL_ADDRESS
msg["To"] = to
msg["Subject"] = subject
msg["Date"] = formatdate(localtime=True)
msg["Message-ID"] = make_msgid(domain=MAIL_ADDRESS.partition("@")[2] or None)
msg.set_content(text) # shown when HTML is not available
msg.add_alternative(html, subtype="html") # what people normally see

token = get_credentials().token
auth_string = f"user={MAIL_ADDRESS}\x01auth=Bearer {token}\x01\x01"

with smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=30) as smtp:
    smtp.starttls(context=ssl.create_default_context()) # 1. encrypt first
    smtp.auth("XOAUTH2", lambda challenge=None: auth_string if challenge is None else "")
    smtp.send_message(msg) # 3. send

```

The message. `set_content` + `add_alternative` build a **multipart/alternative** email: the same message as plain text and as HTML. The richest version must come last, or many clients show raw HTML. Date and Message-ID are set because mail without them looks machine-generated and lands in Spam more often.

The SMTP conversation, in this exact order:

- **Connect** to `smtp.gmail.com:587`. The `with` block closes the connection (QUIT) automatically, even after an error.
- **STARTTLS** - upgrade to an encrypted connection. It must come *before* the login: Gmail does not even offer a login until the connection is encrypted.
- **AUTH XOAUTH2** - log in. Instead of a password, the client sends `user=<email>` and `auth=Bearer <token>`, separated by the control character `\x01`. `smtplib` base64-encodes it.
- **MAIL FROM / RCPT TO / DATA** - `send_message` sends the envelope and the message.

The small `lambda` answers the server. On the first call (no challenge) it returns the login string. If Google rejects the token it sends back a challenge containing an error, and the `lambda` answers with an empty line, so the login ends with a clear 535 error instead of hanging.

Base64 is not encryption. Anybody could decode the login string. It is safe only because STARTTLS already encrypted the connection - which is why the order of the two lines matters.

Block 5 - the email with the buttons

```
def send_request_to_manager(req):
    link = f"{BASE_URL}/nghiphep/{req.id}/decision"
    yes_url = f"{link}?answer=yes&token={req.token}"
    no_url = f"{link}?answer=no&token={req.token}"
    ...
    send(req.manager_email,
         f"[ABSENT #{req.id}] Absence request from {req.employee_email}", text, html)
```

Two different tokens appear in this project - do not mix them up:

- **The OAuth token** (Block 4) proves to Gmail that the *app* may send mail.
- **The request token** (`req.token`, here) proves to our API that the *manager* received the email. Both links carry the same one; only answer differs. There is no login - holding the link is the proof.

`BASE_URL` must be reachable **from the manager's device**. With `localhost` the buttons only work on the computer running the server.

3. main.py

The four routes. Everything the outside world can call.

Block 1 - validation you do not have to write

```
class AbsenceIn(BaseModel):
    employee_email: EmailStr
    manager_email: EmailStr
    reason: str = Field(min_length=1, max_length=500)
    from_date: date
    to_date: date
```

Pydantic enforces all of this **before your function runs**. A malformed address, an empty reason, a 900-character reason or a date like 2026-13-45 never reaches `create_absence` - FastAPI answers **422** automatically, naming the bad field.

- EmailStr needs the email-validator package - that is why it is in `requirements.txt`.
- This class also generates the form at `/docs`. The documentation cannot drift from the code, because it *is* the code.

Block 2 - what the API shows

```
def as_json(req):
    return {
        "id": req.id, "status": req.status,
        "employee_email": req.employee_email, "manager_email": req.manager_email,
        "from_date": req.from_date, "to_date": req.to_date, "reason": req.reason,
    }
```

Look at what is missing: token. The fields are listed by hand precisely so the secret cannot leak by accident. Returning the whole object would have included it, and anybody able to create a request could then approve it instantly.

Block 3 - the employee asks

```
@app.post("/nghiphep", status_code=201)
def create_absence(body: AbsenceIn):
    employee = body.employee_email.lower()
    manager = body.manager_email.lower()

    if not store.is_approved(employee, "employees"):
        raise HTTPException(403, f"{employee} is not on the approved employee list")
    if not store.is_approved(manager, "managers"):
        raise HTTPException(403, f"{manager} is not on the approved manager list")

    req = store.add(employee, manager, body.reason, str(body.from_date), str(body.to_date))
    mail.send_request_to_manager(req)
    return {"id": req.id, "status": req.status, "message": "..."}


```

Look at the **order**: check, then save, then email. The approved list is checked *before* anything else, so a refused request leaves no trace - nothing stored, no email sent. 403 Forbidden is the right answer: the request was valid, but this person is not allowed to make it.

Addresses are lower-cased once, at the entrance, so every comparison after this point is reliable. `status_code=201` means *Created*.

Block 4 - the manager answers

```
@app.get("/nghiphep/{request_id}/decision", response_class=HTMLResponse)
def decide(request_id: int, answer: str, token: str):
    req = store.get(request_id)

    if (req is None or answer not in ("yes", "no"))
```



```

        or not hmac.compare_digest(token, req.token)):
    return page("Invalid link", "This link is not valid.", "#d93025", 400)

if req.status != "PENDING":
    return page("Already answered", ..., "#5f6368")

if datetime.now(timezone.utc) - req.created_at > timedelta(hours=48):
    return page("Link expired", ..., "#f29900", 410)

updated = store.decide(request_id, answer == "yes")
if updated is None:
    return page("Already answered", ..., "#5f6368")

mail.send_result_to_employee(updated)

```

This function is the security core of the project. Four things to notice:

- **Unknown request, silly answer and wrong token give the SAME page.** Telling them apart would help somebody guessing links - they would learn which part they got right.
- `hmac.compare_digest` **instead of** `==`. Python's `==` stops at the first differing character, so a guess sharing the first ten characters takes measurably longer. Measuring those microseconds reveals the token one letter at a time - a *timing attack*. `compare_digest` always takes the same time.
- **Two "already answered" checks are both needed.** The first is the friendly path for a second click minutes later. The last closes the race where two clicks arrive in the same instant - only one can win inside the lock in `store.py`.
- **48 hours** bounds the damage of an old forwarded email.

A GET request that changes data is not normally good practice, but a button in an email can only produce a GET. The honest limitation: some corporate mail scanners open links automatically, which would answer the request by itself. Production would show a confirmation page first.

Block 5 - reading the status

```

@app.get("/nghiphep")
def list_absences():
    return [as_json(req) for req in store.all_requests()]

@app.get("/nghiphep/{request_id}")
def get_absence(request_id: int):
    req = store.get(request_id)
    if req is None:
        raise HTTPException(404, "Absence request not found")
    return as_json(req)

```

Both go through `as_json`, so neither can ever expose the token. `HTTPException(404, ...)` is how FastAPI turns a missing record into a proper HTTP answer rather than a crash.

4. SMTP compared with the Gmail API

Two ways to send the same email from code - and why this demo chose SMTP.

Both send the **same MIME email** from the **same mailbox**, and both log in with an **OAuth 2.0 token** from the same Google Cloud project. They differ in *how* the email is handed to Gmail:

SMTP (this demo)

```
app --SMTP, port 587, STARTTLS, AUTH XOAUTH2--> smtp.gmail.com --> recipient
```

Gmail API

```
app --HTTPS POST, "Authorization: Bearer <token>"--> gmail.googleapis.com --> recipient
```

In both cases Gmail then delivers to the recipient's server using SMTP.

	SMTP + XOAUTH2	Gmail API
Protocol	SMTP (the email standard)	HTTPS / REST (Google's)
Server	smtp.gmail.com:587	gmail.googleapis.com:443
Encryption	STARTTLS	HTTPS (TLS)
Login	AUTH XOAUTH2 + token	Authorization: Bearer token
Smallest scope allowed	https://mail.google.com/ (FULL mailbox)	gmail.send (send only)
Works with other hosts	yes - every mail server speaks SMTP (login differs)	no - Gmail only
Python library	smtplib (built in)	google-api-python-client
Often blocked by	school / company Wi-Fi (port 587)	rarely (it is just HTTPS)
Errors	SMTP codes (535, 550 ...)	JSON with explanations
Extra features	send only	labels, drafts, search ...

Why SMTP here

- It is the **standard** every mail system speaks. The same sending code works with other providers; only the host and the login (their own token or password) change.
- It shows the classic protocol steps on screen: connect, STARTTLS, AUTH, send.
- It still uses **Google Cloud and OAuth 2.0** for the login, so no password is stored.

What it costs

- **Least privilege is weaker.** Google requires the full-mail scope for SMTP, so a stolen token.json could read and delete mail, not only send. The Gmail API version could be limited to gmail.send.
- **Network sensitivity.** Some networks block outgoing port 587; the Gmail API uses normal HTTPS on port 443.

Honest one-liner for class: "We use SMTP, the standard protocol, and log in with an OAuth 2.0 token instead of a password. The trade-off is that Google requires full-mail access for SMTP; the Gmail API would allow send-only, which is the more secure choice in production."

Appendix - the flow through the files

One request, traced from Swagger to the employee's inbox.

```
1. main.py      POST /nghiphep      Pydantic has already validated the body
2. store.py     is_approved() x 2    not on the list? -> 403, nothing saved or sent
3. store.py     add()              token = secrets.token_urlsafe(32)
4. mail.py      send_request_to_... builds ?answer=yes&token=<request token>
5. mail.py      render template    autoescape makes <script> harmless
6. mail.py      get_credentials()   OAuth token: valid / refresh / raise
7. mail.py      send()              SMTP: connect 587 -> STARTTLS -> AUTH XOAUTH2 -> send
8.              -> smtp.gmail.com -> the manager's inbox
9. main.py      returns id + status the request token is NOT in the response

... the manager clicks [Yes, approve] ...

10. main.py     GET .../decision    exists? valid answer? token matches?
11. main.py     still PENDING? younger than 48 hours?
12. store.py     decide()            inside the lock: check, then write
13. mail.py      send_result_to_...  SMTP again -> the employee's inbox
14. main.py      "Request approved"  page shown in the manager's browser
```

Six ideas worth remembering

- **Secrets live in .env and token.json, never in the code.**
- **Only approved people take part**, each in their own role, checked before anything is saved or sent.
- **The request token is random, secret, single-use and expiring**, and the API never returns it. That is the entire authentication for the manager.
- **Check first, then act, and hold a lock across both.** It is what makes the Yes/No button work exactly once.
- **SMTP order matters: STARTTLS before AUTH.** The login string is only base64, so it must travel inside an encrypted connection.
- **Google Cloud issues the permission, not the email.** The app is registered there, the owner approves once, and the OAuth token replaces the password in the SMTP login. An app password would have been a permanent secret for the whole mailbox.

Generated from the source of the app/ package.